

TUIpeRacer2 — Relatório Técnico

Jogo de Corrida de Digitação em JavaFX

1. Visão Geral

O **TUIpeRacer2** é um jogo de corrida de digitação em Java/JavaFX onde jogadores competem digitando um texto o mais rápido e corretamente possível. Suporta três tipos de jogador: **Local** (humano), **CPU** (IA com WPM configurável) e **TCP** (remoto via rede). A arquitetura segue o padrão **MVC** com camadas de modelo, serviço, persistência, rede e view.

2. Diagrama UML Simplificado

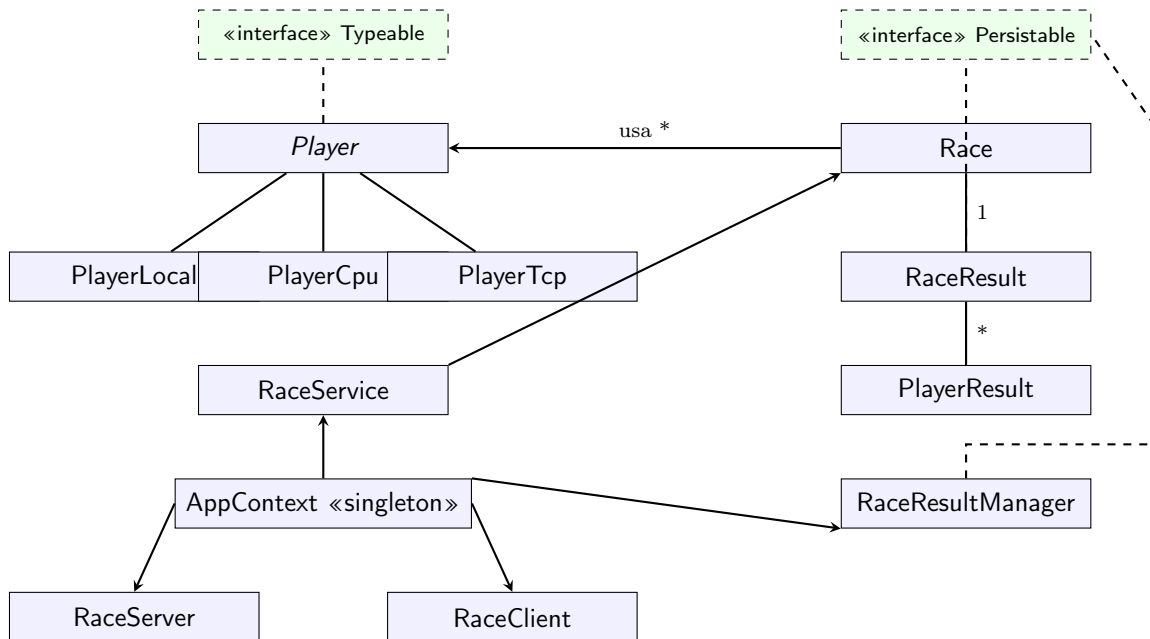


Figura 1: Diagrama UML — hierarquia, interfaces e relacionamentos

3. Hierarquia de Classes

Player (abstrata, implementa **Typeable**) — raiz da hierarquia. Centraliza atributos comuns: texto a digitar, texto digitado, WPM, acurácia e timestamps. Define **processInput(char)** como abstrato.

- **PlayerLocal** — jogador humano; valida o caractere digitado e lança **InvalidInputException** em caso de erro.
- **PlayerCpu** — IA com WPM-alvo e jitter; auto-digita via **autoTick(long nowMs)**.
- **PlayerTcp** — espelho de jogador remoto; recebe caracteres da rede.

Race — encapsula estado da corrida (**WAITING** / **RUNNING** / **FINISHED**) e a lista de jogadores. **RaceResult** (implementa **Persistable**) — agrega resultados individuais (**PlayerResult**) e suporta serialização.

AppContext (*Singleton*) — container de serviços: **RaceService**, **RaceResultManager**, **RaceServer** e **RaceClient**.

4. Interfaces

Typeable — contrato de digitação, implementado por toda a hierarquia `Player`:

```
1 interface Typeable {
2     void processInput(char c)
3         throws InvalidInputException;
4     boolean isFinished();
5     double getProgress(); // 0.0-1.0
6     double getWPM();
7     double getAccuracy(); // 0-100
8 }
```

Persistable — contrato de serialização, implementado por `RaceResult` e `RaceResultManager`:

```
1 interface Persistable {
2     String serialize();
3     void deserialize(String data)
4         throws FileLoadException;
5 }
```

5. Exceções Próprias

Pacote `com.tuiperacer.exception` — distingue exceções *checked* e *unchecked*:

Exceção	Tipo	Quando é lançada
<code>FileLoadException</code>	Checked	Falha na leitura de arquivo de texto ou histórico
<code>PlayerNotFoundException</code>	Checked	Busca de jogador inexistente na corrida
<code>DuplicatePlayerException</code>	Checked	Nome de jogador duplicado ao adicionar
<code>ConnectionException</code>	Checked	Falha na conexão TCP
<code>InvalidTextException</code>	Checked	Texto da corrida vazio ou inválido
<code>RaceAlreadyStartedException</code>	Runtime	Modificação de corrida em andamento
<code>InvalidInputException</code>	Runtime	Caractere digitado diferente do esperado

6. Fluxo de Exceção entre Camadas

Cenário: usuário inicia corrida sem definir texto. A exceção percorre três camadas:

1. **Model** — `Race.start()` detecta texto vazio e lança `InvalidTextException`.
2. **Service** — `RaceService.startRace()` propaga a exceção (declarada em assinatura).
3. **View** — `SetupController` captura e exibe `Alert` ao usuário:

```
1 // Race.java (Model)
2 public void start() throws InvalidTextException {
3     if (text == null || text.isBlank())
4         throw new InvalidTextException("Texto nao pode ser vazio.");
5     state = RaceState.RUNNING;
6 }
7
8 // SetupController.java (View)
9 try {
10     raceService.startRace(); // Service propaga
11     Main.navigateTo("race");
12 } catch (InvalidTextException e) {
13     new Alert(Alert.AlertType.ERROR, "Defina um texto para a corrida.").show();
14 }
```

7. Coleções Utilizadas

Coleção	Local	Finalidade
<code>ArrayList<Player></code>	<code>Race</code>	Lista polimórfica de jogadores
<code>ArrayList<PlayerResult></code>	<code>RaceResult</code>	Resultados individuais da corrida
<code>ArrayList<RaceResult></code>	<code>RaceResultManager</code>	Histórico persistido em arquivo
<code>ConcurrentHashMap<Integer, ClientHandler></code>	<code>RaceServer</code>	Clientes TCP thread-safe
<code>HashMap<String, PlayerPane></code>	<code>RaceController</code>	Lookup de painel UI por nome
<code>ConcurrentLinkedQueue<RemoteChar></code>	<code>RaceController</code>	Fila não-bloqueante de eventos de rede

O `ArrayList<Player>` em `Race` é central: permite iterar polimorficamente sobre todos os tipos de jogador. O `ConcurrentLinkedQueue` desacopla a thread de rede do `AnimationTimer` JavaFX sem bloqueios.

8. Interface Gráfica

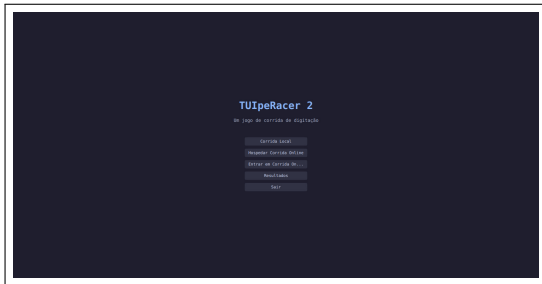


Figura 2: *
Menu Principal

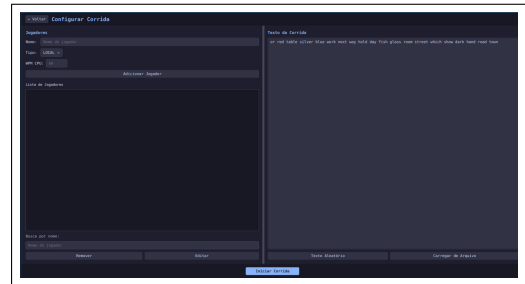


Figura 3: *
Configuração de Corrida

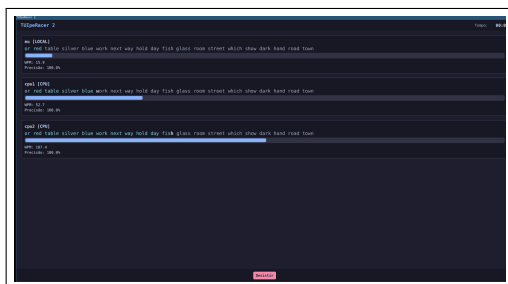


Figura 4: *
Corrida em Andamento



Figura 5: *
Tela de Resultados